

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Exercícios - Tuplas

Exercício 1: Dados Pessoais

```
# Crie uma tupla com seus dados pessoais
# (nome, idade, profissão, cidade)
# Acesse cada elemento e exiba as informações
```

```
# Solução:
dados_pessoais = ("João Silva", 25, "Programador", "Salvador")
```

```
# Acessando elementos
nome = dados_pessoais[0]
idade = dados_pessoais[1]
profissao = dados_pessoais[2]
cidade = dados_pessoais[3]
```

```
print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Profissão: {profissao}")
print(f"Cidade: {cidade}")
```

```
# Desempacotamento
nome, idade, profissao, cidade = dados_pessoais
print(f"Desempacotamento: {nome} tem {idade} anos")
```



Exercício 2: Coordenadas Geográficas

```
# Crie tuplas representando coordenadas de cidades
# Calcule a distância entre duas cidades
```

```
# Solução:
salvador = (-12.9714, -38.5014)
sao_paulo = (-23.5505, -46.6333)
```

```
# Desempacotando coordenadas
lat1, lon1 = salvador
lat2, lon2 = sao_paulo
```

```
# Calculando distância (fórmula simplificada)
```



```
distancia = ((lat2 - lat1)**2 + (lon2 - lon1)**2)**0.5
print(f"Distância Salvador-SP: {distancia:.2f} graus")
```

```
# Criando tupla de resultado
resultado = (distancia, "graus")
print(f"Resultado: {resultado}")
```

Exercício 3: Sistema de Notas

```
# Crie uma tupla com notas de um aluno
# Calcule a média e determine se foi aprovado
```

```
# Solução:
notas = (8.5, 7.0, 9.2, 6.8, 8.0)
```

```
# Calculando média
soma = sum(notas)
media = soma / len(notas)
```

```
# Determinando status
status = "Aprovado" if media >= 7.0 else "Reprovado"
```

```
print(f"Notas: {notas}")
print(f"Média: {media:.2f}")
print(f"Status: {status}")
```

```
# Usando funções built-in
print(f"Maior nota: {max(notas)}")
print(f"Menor nota: {min(notas)}")
print(f"Total de notas: {len(notas)}")
```



Exercício 4: Análise de Dados

```
# Crie uma tupla com dados de vendas mensais
# Analise os dados e encontre o melhor mês
```

```
# Solução:
vendas = (15000, 18000, 12000, 22000, 19000)
meses = ("Janeiro", "Fevereiro", "Março", "Abril", "Maio")
```

```
# Encontrando melhor mês
maior_venda = max(vendas)
indice_melhor = vendas.index(maior_venda)
melhor_mes = meses[indice_melhor]
```

```
# Estatísticas
total_vendas = sum(vendas)
media_vendas = total_vendas / len(vendas)
```

```
print(f"Vendas: {vendas}")
print(f"Melhor mês: {melhor_mes} (R$ {maior_venda})")
print(f"Total: R$ {total_vendas}")
```



```
print(f"Média: R$ {media_vendas:.2f}")

# Usando zip para combinar dados
dados_combinados = list(zip(meses, vendas))
print(f"Dados combinados: {dados_combinados}")
```



Exercício 5: Sistema de Pontuação

```
# Crie um sistema de pontuação para um jogo
# Calcule estatísticas dos jogadores

# Solução:
pontuacoes = (100, 150, 200, 150, 300, 150, 250)

# Análise das pontuações
total_pontos = sum(pontuacoes)
pontuacao_maxima = max(pontuacoes)
pontuacao_minima = min(pontuacoes)
media_pontos = total_pontos / len(pontuacoes)

# Contando pontuações específicas
qtd_150 = pontuacoes.count(150)
pos_300 = pontuacoes.index(300)

print(f"Pontuações: {pontuacoes}")
print(f"Total: {total_pontos}")
print(f"Máxima: {pontuacao_maxima}")
print(f"Mínima: {pontuacao_minima}")
print(f"Média: {media_pontos:.2f}")
print(f"Quantas vezes 150: {qtd_150}")
print(f"Posição do 300: {pos_300}")

# Criando tupla de estatísticas
estatisticas = (total_pontos, pontuacao_maxima, pontuacao_minima, media_pontos)
print(f"Estatísticas: {estatisticas}")
```



Exercício 6: Tupla Aninhada

```
# Crie uma tupla contendo outras tuplas
# Represente pontos em um plano cartesiano

# Solução:
pontos = (
    (0, 0),      # Origem
    (3, 4),      # Ponto A
    (6, 8),      # Ponto B
    (9, 12),     # Ponto C
)

print("Pontos:", pontos)

# Calculando distâncias da origem
```

```
for i, ponto in enumerate(pontos):
    x, y = ponto
    distancia = (x**2 + y**2)**0.5
    print(f"Ponto {i}: {ponto} - Distância da origem: {distancia:.2f}")

# Encontrando ponto mais distante
distancias = []
for ponto in pontos:
    x, y = ponto
    distancia = (x**2 + y**2)**0.5
    distancias.append(distancia)

maior_distancia = max(distancias)
indice_maior = distancias.index(maior_distancia)
ponto_mais_distante = pontos[indice_maior]

print(f"Ponto mais distante: {ponto_mais_distante} (distância: {maior_distancia:.2f})")
```



Exemplo: Sistema de Coordenadas

```
# Sistema de coordenadas para um jogo
coordenadas_jogo = (
    (0, 0),      # Spawn
    (10, 5),     # Checkpoint 1
    (20, 10),    # Checkpoint 2
    (30, 15),    # Checkpoint 3
    (40, 20),    # Final
)

# Calculando distância total do percurso
distancia_total = 0
for i in range(1, len(coordenadas_jogo)):
    x1, y1 = coordenadas_jogo[i-1]
    x2, y2 = coordenadas_jogo[i]
    distancia = ((x2 - x1)**2 + (y2 - y1)**2)**0.5
    distancia_total += distancia

print(f"Distância total do percurso: {distancia_total:.2f} unidades")
```



Dicas para os Exercícios

- **Desempacotamento:** Use para acessar elementos facilmente
- **Funções built-in:** Aproveite max(), min(), sum(), len()
- **Métodos:** Use count() e index() quando necessário
- **Fatiamento:** Extraia partes da tupla quando útil

